

Kubernetes 企業實戰教育訓練

實戰部署工作坊

7 個真實開源應用 · 一座正式 K8s 叢集 · 難度逐站遞增

主講人：李旻恩

[draw.io](#)

[filebrowser](#)

[flarum](#)

[planka](#)

[onlyoffice](#)

[peertube](#)

[cloudbeaver](#)



Opc Ua Extender

opc UA



USB UltraLock



今天的路線

01

環境與慣例

叢集架構、每個產品的四種形式、所有產品共用的標準骨架

02

七站實作

從無狀態的 draw.io 一路到跨 namespace 的 cloudbeaver，每站只多一點新東西

03

練習與驗收

共通教訓、填空題與除錯題、四個綠燈的驗收標準

專案：<https://github.com/AnnLeemocha/k8s-training>

01

環境與慣例

單元 1 叢集架構、每個產品的四種形式、所有產品共用的標準骨架



上課前要知道的四件事



沒有 Ingress，一律 Gateway API

Cilium 實作。七個產品全部用 HTTPRoute，沒有 NodePort 或 Ingress 範例。



沒有 default StorageClass

每個 PVC 都要明確指定：rook-ceph-block (RWO) 或 rook-cephfs (RWX)。



注意資源，若記憶體吃緊

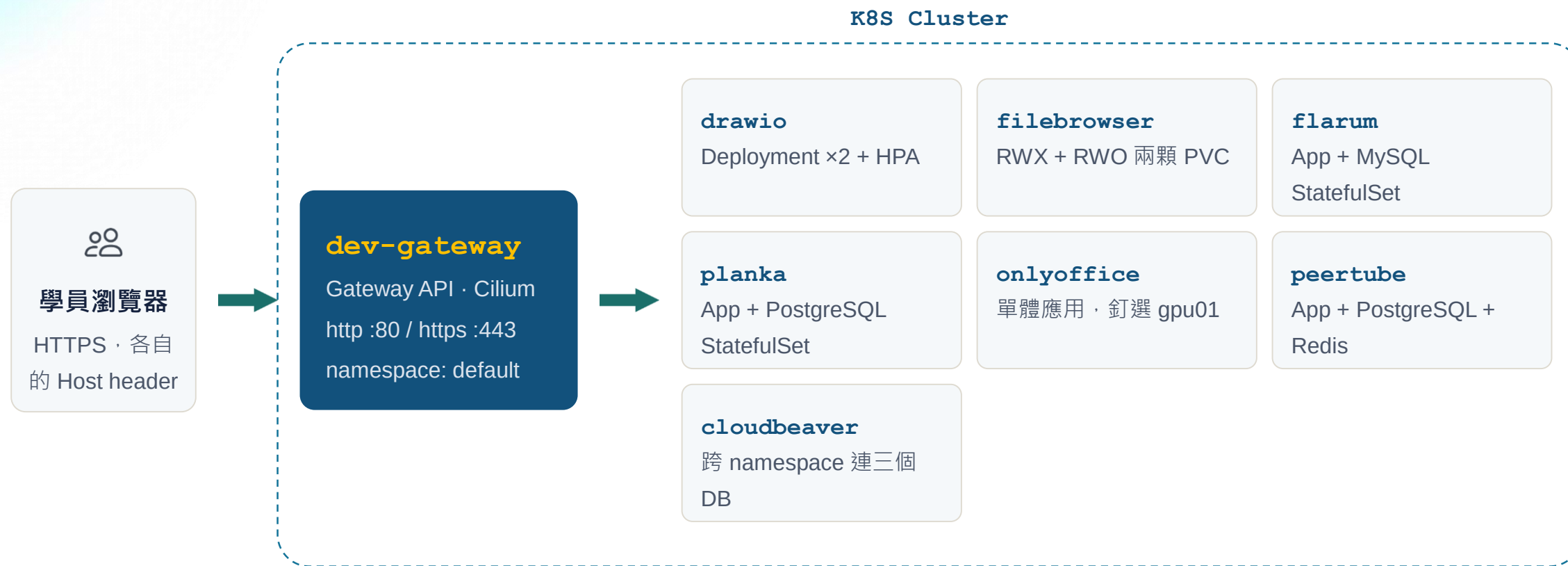
此範例中，資源消耗較大的產品部屬到帶有汙點 (taint) 節點 (gpu0X) 上，實際根據需求部屬。



一產品一 namespace

彼此用 NetworkPolicy 隔離。TLS 是貴公司提供的 *.nexai.org.com，瀏覽器應可解析，不會出現警告。

叢集架構：一個入口，七個 namespace



一個產品一個 namespace，可以決定這些資源屬於誰、整體最多能用多少。刪掉 namespace，裡面的東西會一起消失。

建立入口：TLS Secret 與 HTTPS

```
# dev-gateway.yaml
apiVersion: gateway.networking.k8s.io/v1
kind: Gateway
metadata:
  name: dev-gateway
  namespace: default
  labels:
    app.kubernetes.io/part-of: k8s-training
    app.kubernetes.io/environment: development
spec:
  gatewayClassName: cilium # Gateway Controller
  listeners:
    - name: http
      port: 80
      protocol: HTTP
      allowedRoutes:
        namespaces:
          from: All
    - name: https
      port: 443
      protocol: HTTPS
      tls:
        mode: Terminate
        certificateRefs:
          - kind: Secret
            name: gateway-tls # 已寫入公司憑證
            namespace: default
      allowedRoutes:
        namespaces:
          from: All
```

TLS 在 Gateway 終止

瀏覽器到 Gateway 走 HTTPS，Gateway 到 Pod 是一般 HTTP，應用不用管憑證。

TLS Secret 跟 Gateway 放一起

gateway-tls 在 default namespace，內含 tls.crt 與 tls.key。

公司憑證 *.nexai.org.com

公司應該已做好解析，瀏覽器不應出現警告。(除非憑證為自簽，curl 要加 -k)

```
# 取得 gatewayclass
kubectl get gatewayclass

# 確認 tls certificate 已建立
kubectl get secret -n default

# 共用的教學 Gateway，整門課只套一次
kubectl apply -f dev-gateway.yaml

# 查看 12 配予的外部 IP
kubectl get gateway
```

標準骨架：每一站都從這些資源開始

1. Namespace

一產品一個，是隔離的邊界，但非防火牆

4. HTTPRoute

掛到 dev-gateway，依 Host 分流
xxx.nexai.org.com 的請求要送去
哪個 Service？

3. Service

ClusterIP，用 label 選中 Pod
要找什麼標籤的 Pod？走哪個 port？

2. Deployment · StatefulSet

副本與掛載，及 Pod 設定
Pod 要跑幾份、用哪個 image、資源要用多少、probes 如何設置、怎麼換版？



5. Quota / LimitRange

管住整個 namespace 的資源用量
最多能用多少資源？



6. NetworkPolicy

Pod 間的防火牆，逐條放行
需要放行誰可以連進\出容器？



Pod

Pod

依產品需要再疊加：

+ PVC

哪些資料要留下？
放哪種儲存？

+ ConfigMap · Secret

哪些資料要留下？放哪種儲存？

+ HPA

需不需要自動擴充？

...

先畫架構，再開始部署

1	使用者從哪個網址進來？	HTTPRoute
2	流量最後進到哪個 port？	Service → Pod
3	哪些資料要留下？放哪種儲存？	PVC · RWO / RWX
4	誰可以連誰？	NetworkPolicy
5	密碼和設定放哪？	Secret · ConfigMap



架構圖上答不出來的問題，
YAML 也寫不對。

每個產品都附
exercises/architecture.drawio · 用
draw.io 本身就能打開。

資源設定：兩層數字要對得上

LimitRange.min

50m / 64Mi (單一容器的資源下限)

≤

requests

100m / 256Mi

≤

limits

500m / 512Mi

≤

LimitRange.max

1 / 1Gi (單一容器的資源上限)

Σ replicas × requests

HPA 最多 5 個 Pod × 100m = 500m

≤

ResourceQuota

requests.cpu 抓 1 (1000m) · 留一倍餘裕

CPU 可壓縮

超過 limit 只會被節流變慢 · 比例可以抓寬 (本例 5 倍)

Memory 不可壓縮

超過 limit 直接 OOMKilled · 比例要抓保守 (本例 2 倍)

02

七站實作

單元 2 從無狀態的 draw.io 一路到跨 namespace 的 cloudbeaver，每站只多一點新東西

教學只教前幾站，其他請學員自行練習



七站路線：產品說明

站	產品	功能
1	draw.io	是開源的圖表繪製工具，可用於繪製架構圖、流程圖、心智圖等，本身是純前端應用
2	filebrowser	網頁版檔案管理器，提供瀏覽、上傳、下載、預覽，並整合 onlyoffice 做線上編輯 Word / Excel / PPT
3	flarum	是極簡、輕量且現代化的開源討論區/論壇軟體
4	planka	是開源、輕量的看板式專案管理工具
5	onlyoffice	是強大的線上 Office 辦公套件，支援 Word、Excel、PPT 的線上即時共同編輯，可與 Nextcloud、Filebrowser 等系統整合
6	peertube	是基於 ActivityPub 協定與 P2P 技術的去中心化影音分享平台（開源版 YouTube）
7	cloudbeaver	是 DBeaver 官方推出的 Web 版本，是一個通用的視覺化資料庫管理工具，同一個瀏覽器介面可以建立多組資料庫連線、支援 MySQL/PostgreSQL 等多種資料庫

七站路線：難度逐站遞增

站	產品	難度	這一站新出現的概念
1	draw.io	★☆☆☆☆	無狀態骨架：Deployment、Service、HTTPRoute、HPA
2	filebrowser	★★☆☆☆	PVC 與 StorageClass (RWX vs RWO)、ConfigMap
3	flarum	★★★☆☆	StatefulSet、initContainer、Secret 互相引用
4	planka	★★★☆☆	跟 flarum 對照：非 root 映像、免 initContainer
5	onlyoffice	★★★★☆	資源規劃 (nodeSelector + toleration)
6	peertube	★★★★★	三元件整合、PVC vs emptyDir、7 條 NetworkPolicy
7	cloudb Beaver	★★★★☆	跨 namespace 存取 (namespaceSelector)

前四站涵蓋全部新概念，講師逐一深講；後三站是既有概念的變化型，適合學員照正解自己重做。

每個產品都有四種形式

專案：<https://github.com/AnnLeemocha/k8s-training/tree/main/apps>

manifest/

教學主線

拆成編號 yaml，一支一支
apply，逐步講解每個資源

all-in-one.yaml

快速重建

跟 manifest/ 一致的合併版，
demo 或重建環境用

chart/

Helm 版本

values.yaml 參數化，教完純
yaml 再講重複部署怎麼簡化

exercises/

學員練習

填空題 + 除錯題 + 架構圖，
正解就是 manifest/

```
# 共用的教學 Gateway，整門課只套一次，以 draw.io 為例
kubectl apply -f dev-gateway.yaml
kubectl apply -f apps/01-draw.io/manifest/
kubectl apply -f apps/01-draw.io/drawio-all-in-one.yaml
helm install drawio apps/01-draw.io/chart/ -n drawio
```

本次教學，我們使用 Rancher 進行產品部屬。

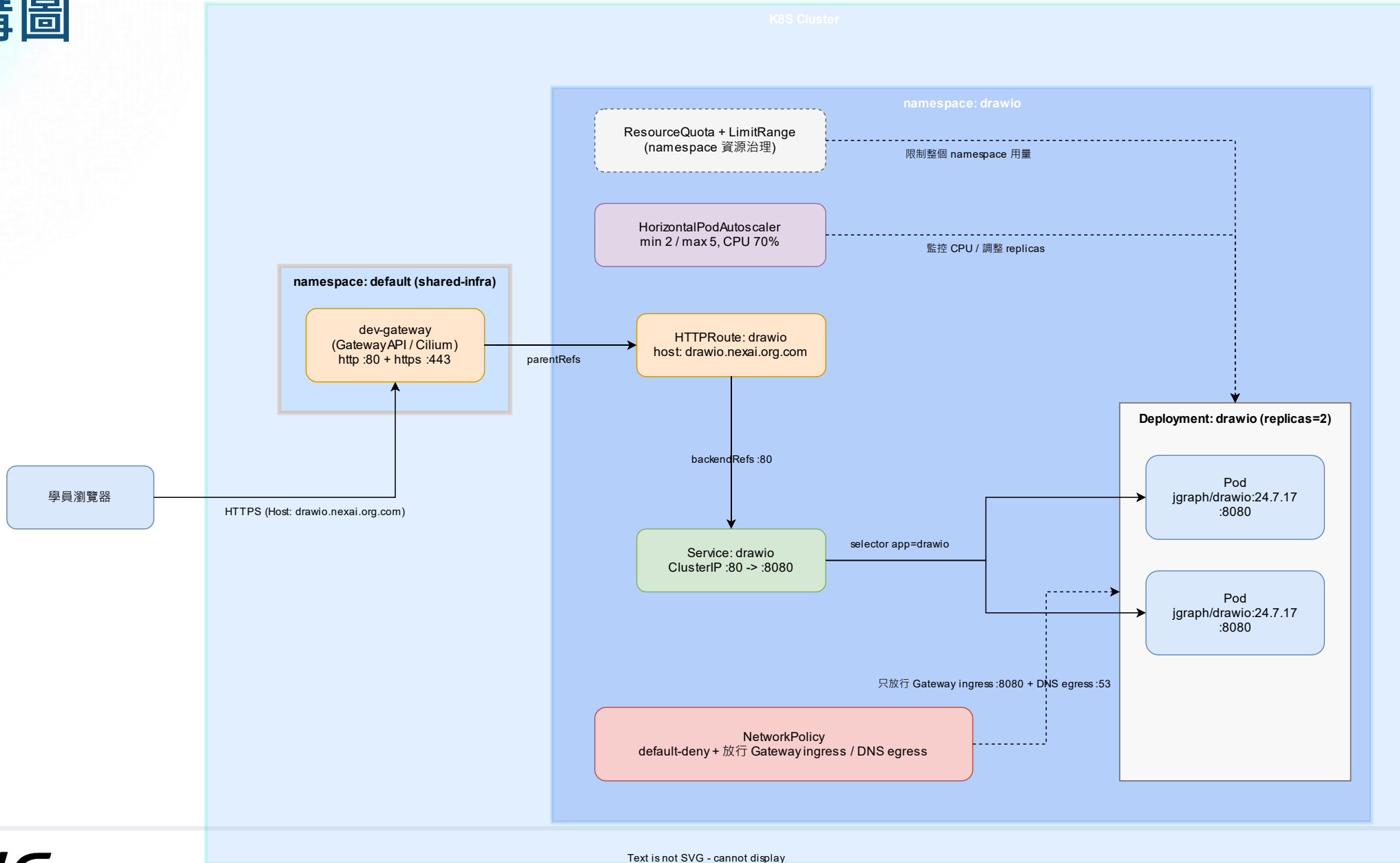
第 1 站 · draw.io

單元 2-1 無狀態的起點

是開源的圖表繪製工具，可用於繪製架構圖、流程圖、心智圖等，本身是純前端的無狀態 Web 應用（jgraph/drawio 映像），不需要資料庫、不需要任何持久化儲存。



第 1 站 · draw.io 架構圖



Namespace & ResourceQuota & LimitRange

```
# 00-namespace.yaml
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: drawio
  labels:
    app.kubernetes.io/part-of: k8s-training
    training/product: drawio
```

```
# 01-resourcequota-limitrange.yaml ①
```

```
apiVersion: v1
kind: ResourceQuota
metadata:
  name: drawio-quota
  namespace: drawio
spec:
  hard:
    requests.cpu: "1"      # 保留量總和
    requests.memory: 1Gi
    limits.cpu: "2"        # 封頂值總和
    limits.memory: 2Gi
    pods: "10"            # Pod 數上限
```

```
# 01-resourcequota-limitrange.yaml ②
```

```
apiVersion: v1
kind: LimitRange
metadata:
  name: drawio-limitrange
  namespace: drawio
spec:
  limits:
    - type: Container
      default:           # 沒寫 limits 時補上
        cpu: 250m
        memory: 256Mi
      defaultRequest:    # 沒寫 requests 時補上
        cpu: 100m
        memory: 128Mi
      max:               # 單一容器上限
        cpu: "1"
        memory: 1Gi
      min:               # 單一容器下限
        cpu: 50m
        memory: 64Mi
```

數字要對得上

$\min \leq \text{requests} \leq \text{limits} \leq \max$; $\sum \text{requests} \leq \text{Quota}$ 。詳見「資源設定」那一頁。

Namespace : 隔離邊界

一個產品一個。labels 只是標記，方便用 `kubectl get ns -l` 篩選。

Quota : 管整個 namespace

所有 Pod 加總不能超過，HPA 擴出來的 Pod 也算在內。

LimitRange : 管單一容器

沒寫 requests / limits 時自動補上預設值；超過 max 或低於 min，apply 直接失敗。

requests vs limits

requests 是排程時保證保留的量；limits 是執行時的封頂值，記憶體超過會被 OOMKilled，CPU 超過則被限流。

Deployment：副本、滾動更新與 Probes

02-deployment.yaml (節錄)

■ 黃字 = 本站新東西

```
spec:
  replicas: 2
  selector:
    matchLabels:
      app: drawio      # 要跟 template 的 labels 一致
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # 更新時先多開 1 個新 Pod
      maxUnavailable: 0 # 更新中不允許少任何 Pod
  template:
    metadata:
      labels:
        app: drawio
    spec:
      containers:
        - name: drawio
          image: jgraph/drawio:24.7.17
          ports:
            - name: http # 讓 probe / Service 用名稱引用
              containerPort: 8080
          resources: ... # 見「資源設定」那一頁
          readinessProbe: # 失敗 → 移出 Service Endpoints
            httpGet: { path: /, port: http }
            initialDelaySeconds: 10
          livenessProbe: # 失敗 → 直接重啟容器
            httpGet: { path: /, port: http }
            initialDelaySeconds: 20
          securityContext:
            allowPrivilegeEscalation: false
            capabilities: { drop: [ALL] }
```

RollingUpdate：換版不停機

maxSurge 1 + maxUnavailable 0：先建新 Pod，等它 Ready 才砍舊的，更新過程服務不中斷。

readiness vs liveness

readiness 失敗只是不送流量；liveness 失敗會重啟容器，所以 liveness 的等待時間要抓得更保守。

具名 port：http

probe 與 Service 都用名稱 http 引用 8080，之後改 port 只要改一個地方。

最小權限 securityContext

禁止提權、收回所有 Linux capabilities。draw.io 不需要任何特權，是最乾淨的範例；後面幾站會看到例外。

Service 與 HTTPRoute：從網址到 Pod

03-service.yaml

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: Service
metadata:
  name: drawio
  namespace: drawio
spec:
  type: ClusterIP      # 只給叢集內部使用
  selector:
    app: drawio        # 選中帶這個 label 的 Pod
  ports:
    - name: http
      port: 80          # Service 自己的 port
      targetPort: http  # 轉到 Pod 的具名 port
```

Service：穩定的入口

Pod 重建會換 IP；Service 用 selector 找 Pod，提供固定的 DNS：
drawio.drawio.svc.cluster.local。

port 對應鏈

HTTPRoute 80 → Service port 80 → targetPort http → containerPort 8080。
任何一環對不上都會連不到。

04-httproute.yaml

```
apiVersion: gateway.networking.k8s.io/v1
kind: HTTPRoute
metadata:
  name: drawio
  namespace: drawio
spec:
  parentRefs:          # 掛到哪個 Gateway 的 listener
    - name: dev-gateway
      namespace: default
      sectionName: http
    - name: dev-gateway
      namespace: default
      sectionName: https
  hostnames:           # 依 Host header 分流
    - "drawio.nexai.org.com"
  rules:
    - matches:
        - path:
            type: PathPrefix
            value: /
        backendRefs:    # 送到哪個 Service、哪個 port
          - name: drawio
            port: 80
```

HTTPRoute 取代 Ingress

七個產品共用同一個 dev-gateway，靠 hostnames 分流。

HPA：依 CPU 自動擴縮

05-hpa.yaml

■ 黃字 = 本站新東西

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: drawio
  namespace: drawio
spec:
  scaleTargetRef:           # 要擴縮哪個工作負載
    apiVersion: apps/v1
    kind: Deployment
    name: drawio
  minReplicas: 2
  maxReplicas: 5           # 要跟 ResourceQuota 一起算
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70  # 相對於 requests
```

HPA 接手 replicas

套用 HPA 後，Deployment 裡的 replicas: 2 只是初始值，實際副本數由 HPA 在 2~5 之間調整。

70% 是相對 requests

平均 CPU 用量超過 requests (100m) 的 70% 就擴容，不是跟節點總 CPU 比。容器沒設 requests，HPA 就算不出來。

maxReplicas 要算進 Quota

5 個 Pod × 100m = 500m ≤ requests.cpu 1。上限開太高，多出來的 Pod 會被 Quota 擋下（下一頁示範）。

觀察指令

kubectl get hpa -n drawio -w
看 TARGETS 欄位（需要 metrics-server，此叢集已安裝）。

觸發 HPA : 用 Job 施壓

```
# hpa-busy-job.yaml
```

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: Namespace
metadata:
  name: hpa-loadgen      # 獨立 namespace
---
apiVersion: batch/v1
kind: Job
metadata:
  name: hpa-loadgen
  namespace: hpa-loadgen
spec:
  activeDeadlineSeconds: 300 # 最多跑 5 分鐘
  backoffLimit: 0           # 失敗不重試
  parallelism: 2            # 同時 2 個施壓 Pod
  completions: 2
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: curl
        image: curlimages/curl:latest
        resources:
          requests: { cpu: 250m, memory: 64Mi }
          limits:   { cpu: "1", memory: 128Mi } # 每個最多 1 核
        command: ["sh", "-c"]
        args:
        - |
          for i in $(seq 1 30); do          # 30 個 curl 迴圈
            (while true; do curl -s -o /dev/null \
              http://drawio.drawio.svc.cluster.local/js/app.min.js; done) &
          done
          wait
```

為什麼放在獨立 namespace

放在 drawio 裡會被 LimitRange 預設的 cpu limit 250m 限流，施壓端自己變成瓶頸；調高 limit 又會吃掉 drawio 的 Quota，HPA 反而開不出新 Pod。drawio 的 ingress 是 from: []，跨 namespace 打得到。

操作

```
# 終端機 A : 觀察
kubectl get hpa drawio -n drawio -w
# 終端機 B : 施壓 (300 秒後自動結束)
kubectl apply -f hpa-busy-job.yaml
# 測完清理
kubectl delete namespace hpa-loadgen
```

實測 : TARGETS 與 REPLICAS

```
07:54:08  cpu:    2%/70%  replicas=2  # 開始施壓
07:54:39  cpu:  109%/70%  replicas=2  # 超過門檻
07:54:54  cpu:  154%/70%  replicas=4  # 一次 2 → 4
07:55:56  cpu:  111%/70%  replicas=5  # 要求加到上限
08:00:30  cpu:    2%/70%  replicas=5  # 縮容要等約 5 分鐘
```

降低 HPA 門檻

根據環境所消耗的資源不同，Pod 若無擴張，可嘗試調整 HPA 門檻或 Pod 的 request 資源

資源不足：Quota 擋下第 5 個 Pod

```
# kubectl get events -n drawio:Deployment 停在 4/5
```

```
FailedCreate ... exceeded quota: drawio-quota,  
requested: limits.cpu=500m,limits.memory=512Mi,requests.memory=256Mi,  
used: limits.cpu=2,limits.memory=2Gi,requests.memory=1Gi
```

Quota 項目	上限	4 個 Pod 已用	第 5 個再加	結果
requests.cpu	1	400m	+100m → 500m	✓ 還夠
requests.memory	1Gi	1Gi	+256Mi → 1.25Gi	✗ 超過
limits.cpu	2	2	+500m → 2.5	✗ 超過
limits.memory	2Gi	2Gi	+512Mi → 2.5Gi	✗ 超過

想讓它真的擴到 5 個

Quota 至少要 requests.memory ≥ 1280Mi、limits.cpu ≥ 2500m、limits.memory ≥ 2560Mi；或把 maxReplicas 改成 4。這裡刻意保留不修，可以當學員練習題。

HPA 只改數字

HPA 把 replicas 設成 5，不保證 Pod 建得出來；ReplicaSet 會一直重試、一直 FailedCreate。

Quota 在建立 Pod 當下檢查

已用量 + 新 Pod 需求，requests 和 limits 都算，任何一項超過就拒絕，Pod 根本不會出現。

分辨兩種「起不來」

沒有 Pod + FailedCreate: exceeded quota → namespace 配額問題。
有 Pod 但 Pending + Insufficient cpu/memory → 節點容量問題。

排查

```
kubectl get deploy drawio -n drawio # READY 4/5  
kubectl get resourcequota -n drawio # 哪幾項滿了  
kubectl get events -n drawio \  
--field-selector reason=FailedCreate
```

NetworkPolicy：先全擋，再逐條放行

06-networkpolicy.yaml ①

```
kind: NetworkPolicy
metadata:
  name: default-deny-all
  namespace: drawio
spec:
  podSelector: {} # 全部 Pod
  policyTypes:
    - Ingress
    - Egress
# 沒有任何規則 = 全擋
```

① 先全擋

空的 podSelector 選中整個 namespace，Ingress、Egress 都列出但不寫規則，等於全部擋下。

06-networkpolicy.yaml ②

```
kind: NetworkPolicy
metadata:
  name: allow-ingress-from-gateway
spec:
  podSelector:
    matchLabels:
      app: drawio
  policyTypes:
    - Ingress
  ingress:
    - from: [] # 不限來源
      ports:
        - protocol: TCP
          port: 8080
```

② 再逐條補洞

放行 Gateway 進來的流量。注意 port 寫的是 Pod 的 8080，不是 Service 的 80。

06-networkpolicy.yaml ③

```
kind: NetworkPolicy
metadata:
  name: allow-egress-dns
spec:
  podSelector: {}
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector: {}
      ports:
        - protocol: UDP
          port: 53
        - protocol: TCP
          port: 53
```

③ 別忘了 DNS

全擋之後連 Service 名稱都解析不了，要對任何 namespace 開 53 port 給 CoreDNS。

第 2 站 · filebrowser

單元 2-2 RWX vs RWO，有狀態

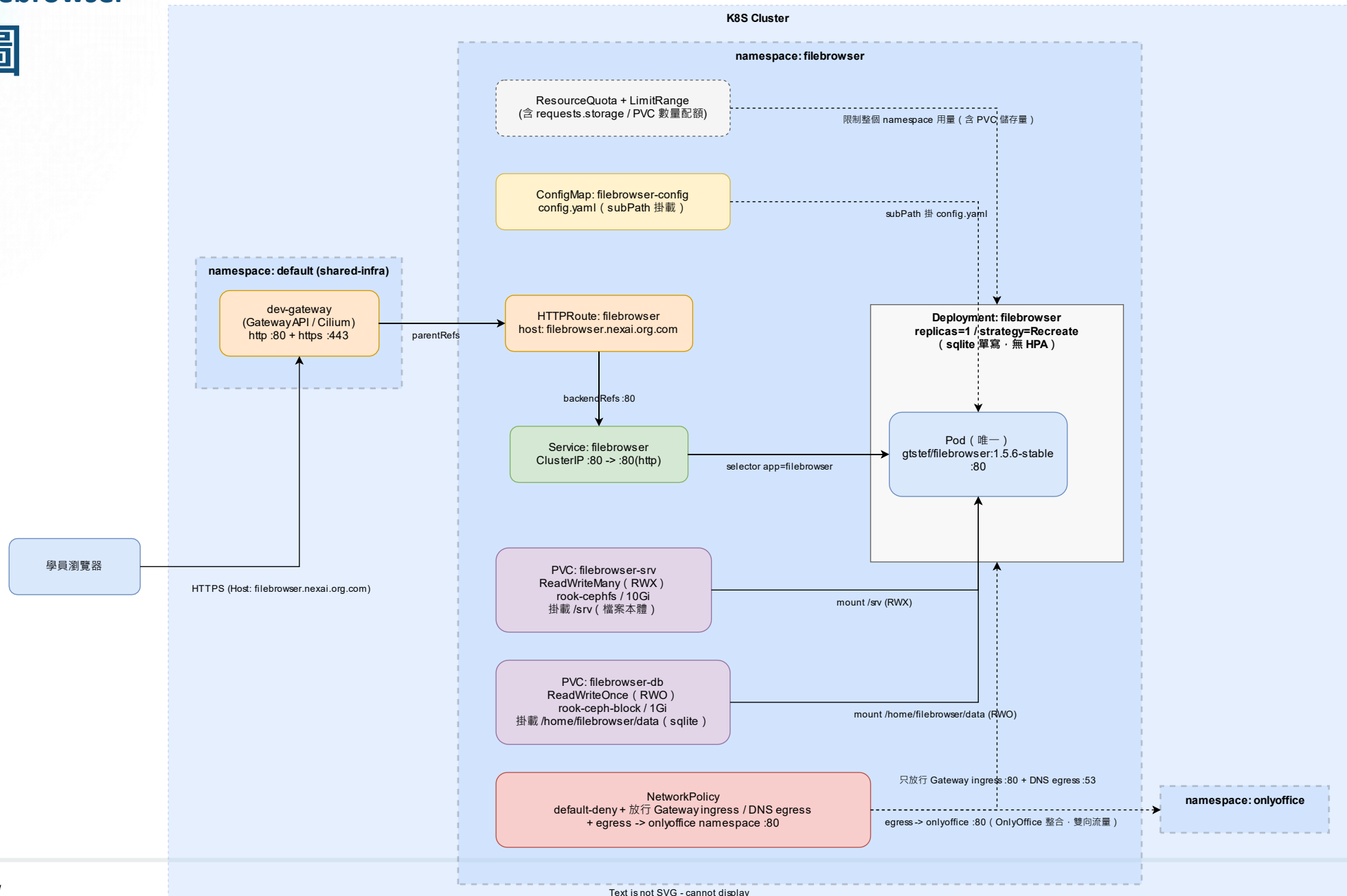
是一個網頁版檔案管理器，提供瀏覽、上傳、下載、預覽，並整合 onlyoffice 做線上編輯 Word / Excel / PPT。

(實際跑的映像是社群 fork FileBrowser Quantum，gtstef/filebrowser:1.5.6-stable)



第 2 站 · filebrowser

架構圖



PVC : RWX 還是 RWO ?

02-pvc.yaml ① 檔案本體

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: filebrowser-srv
  namespace: filebrowser
spec:
  accessModes:
    - ReadWriteMany      # RWX : 多個 Pod 可同時讀寫
  storageClassName: rook-cephfs
  resources:
    requests:
      storage: 10Gi
```

02-pvc.yaml ② sqlite 資料庫 + 快取

```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: filebrowser-db
  namespace: filebrowser
spec:
  accessModes:
    - ReadWriteOnce      # RWO : 同時只給一個節點掛
  storageClassName: rook-ceph-block
  resources:
    requests:
      storage: 1Gi
```

沒有 default StorageClass

storageClassName 一定要寫；漏寫的 PVC 會一直 Pending，Pod 也跟著起不來。

RWX 還是 RWO ?

不是看應用種類，而是看「這份資料允不允許多個 Pod 同時寫」。sqlite 不支援多個行程同時寫入，所以選 RWO。

cephfs vs ceph-block

rook-cephfs 是檔案系統，支援 RWX；rook-ceph-block 是區塊裝置，只支援 RWO，但比較輕量。

ConfigMap 與 subPath：把設定檔疊進 PVC

03-configmap.yaml (節錄)

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: filebrowser-config
  namespace: filebrowser
data:
  config.yaml: |      # key = 檔名 · 內容 = 整份設定檔
    server:
      port: 80
      disableUpdateCheck: true  # 不打外網
      database: "/home/filebrowser/data/filebrowser.db"
    integrations:
      office:
        url: "http://onlyoffice.nexai.org.com"
        internalUrl: "http://onlyoffice.onlyoffice.svc.cluster.local"
```

ConfigMap 掛成檔案

data 的每個 key 變成 volume 裡的一個檔案。disableUpdateCheck：零信任網路連不到 GitHub，不關會卡 31 秒才開機。

Recreate：先砍舊再建新

sqlite 不能讓新舊 Pod 同時寫入，更新會短暫中斷（對照 draw.io 的 RollingUpdate）。

04-deployment.yaml (節錄)

```
spec:
  replicas: 1          # sqlite 不支援多寫者
  strategy:
    type: Recreate
  template:
    spec:
      containers:
        - name: filebrowser
          volumeMounts:
            - name: srv
              mountPath: /srv
            - name: db
              mountPath: /home/filebrowser/data
            - name: config
              mountPath: /home/filebrowser/data/config.yaml
              subPath: config.yaml  # 只疊一個檔案
              readOnly: true
      volumes:
        - { name: srv, persistentVolumeClaim: { claimName: filebrowser-srv } }
        - { name: db, persistentVolumeClaim: { claimName: filebrowser-db } }
        - name: config
          configMap:
            name: filebrowser-config
```

subPath：只疊一個檔案

若把 ConfigMap 直接掛在 /home/filebrowser/data，整個目錄會被蓋掉，sqlite 也跟著不見。

NetworkPolicy：跨 namespace 的 egress

07-networkpolicy.yaml (新增的第 4 條)

■ 黃字 = 本站新東西

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-filebrowser-to-onlyoffice
  namespace: filebrowser
spec:
  podSelector:
    matchLabels:
      app: filebrowser
  policyTypes:
    - Egress # 管「連出去」
  egress:
    - to:
        - namespaceSelector: # 跨 namespace
            matchLabels:
              kubernetes.io/metadata.name: onlyoffice
  ports:
    - protocol: TCP
      port: 80
```

05-onlyoffice/manifest/07-networkpolicy.yaml (對應的一條)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: allow-egress-onlyoffice-to-filebrowser
  namespace: onlyoffice
spec:
  podSelector:
    matchLabels:
      app: onlyoffice
  policyTypes:
    - Egress # 回頭抓檔、回報存檔
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: filebrowser
  ports:
    - protocol: TCP
      port: 80
```

為什麼要多開 egress

filebrowser 要主動呼叫 onlyoffice 的 Document Server；default-deny-all 之後，除了 DNS 其他出站都被擋。

namespaceSelector

比對 K8s 自動加在每個 namespace 上的 kubernetes.io/metadata.name，只放行 onlyoffice 一個 namespace 的 80 port，比 namespaceSelector: {} 精準。

兩邊都要開

onlyoffice 要用 internalUrl 回頭抓檔案，所以它也要一條 egress 連 filebrowser。對方的 ingress 都是 from: []，不用再加。

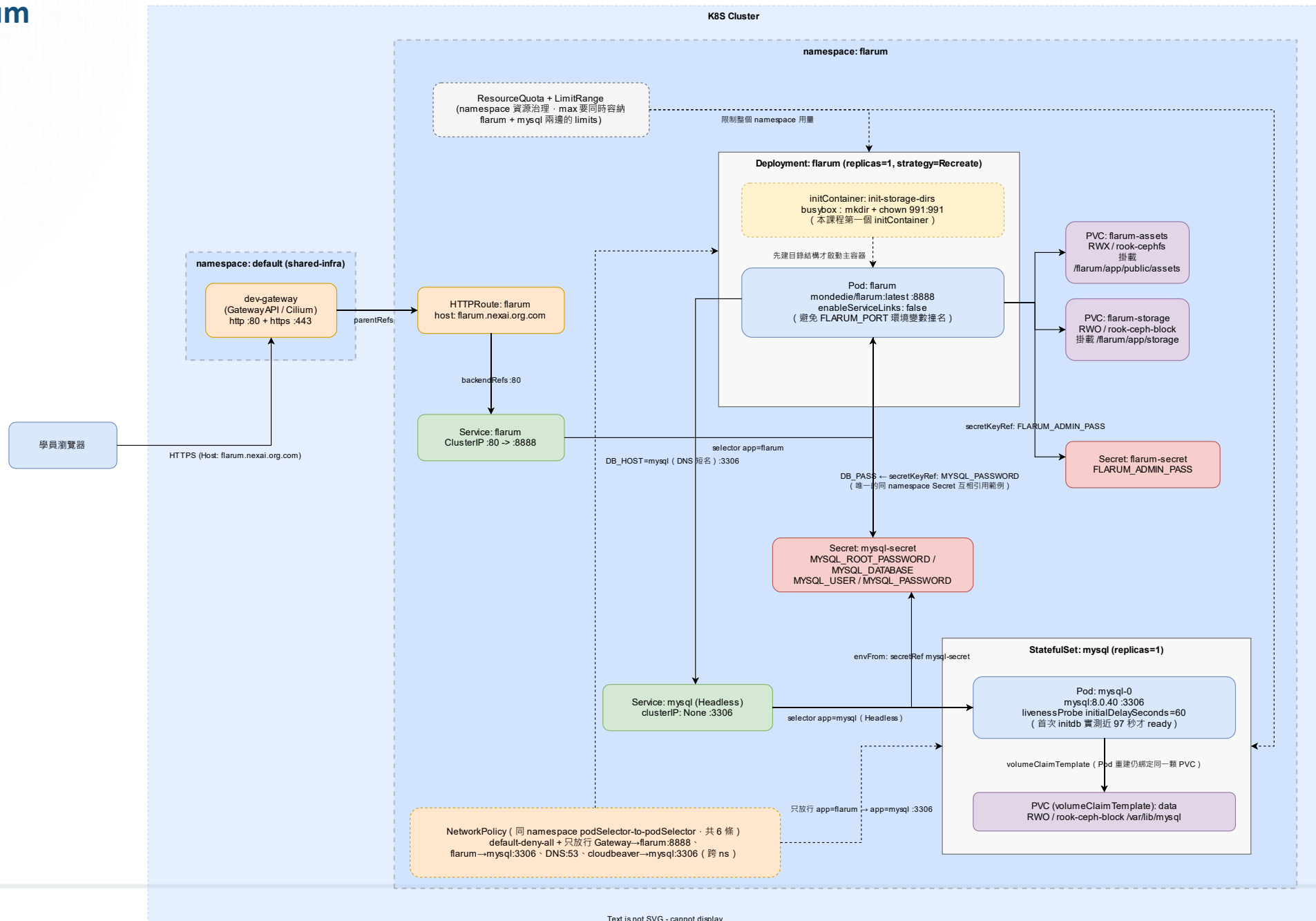
第 3 站 · flarum

單元 2-3 App 與 MySQL 住同一個 namespace

是極簡、輕量且現代化的開源討論區/論壇軟體。([mondedie/flarum](#) 映像)



第3站 · flarum 架構圖



Secret 與 Headless Service

02-mysql-secret.yaml

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
  namespace: flarum
type: Opaque
stringData:      # 寫明文，存進去只是 base64
  MYSQL_ROOT_PASSWORD: "TrainingRoot123!"
  MYSQL_DATABASE: "appdb"
  MYSQL_USER: "appuser"
  MYSQL_PASSWORD: "TrainingApp123!"
```

03-mysql-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
  namespace: flarum
spec:
  clusterIP: None      # Headless : DNS 直接回 Pod IP
  selector:
    app: mysql
  ports:
    - name: mysql
      port: 3306
      targetPort: mysql
```

07-flarum-deployment.yaml (env 節錄)

```
env:
  - name: DB_HOST
    value: "mysql"      # 同 namespace 用短名
  - name: DB_USER
    value: "appuser"
  - name: DB_PASS
    valueFrom:
      secretKeyRef:
        name: mysql-secret  # 別人的
        key: MYSQL_PASSWORD
  - name: FLARUM_ADMIN_PASS
    valueFrom:
      secretKeyRef:
        name: flarum-secret  # 自己的
        key: FLARUM_ADMIN_PASS
```

Secret 互相引用

同一個 namespace 內，Pod 可以從多個 Secret 各取一個 key 來組合。mysql 密碼只存一份，不用複製到 flarum-secret。

Headless Service

clusterIP: None 搭配 StatefulSet，每個 Pod 有固定 DNS：mysql-0.mysql.flarum.svc.cluster.local。

StatefulSet：資料庫的家

04-mysql-statefulset.yaml (節錄)

■ 黃字 = 本站新東西

```
kind: StatefulSet
metadata:
  name: mysql
  namespace: flarum
spec:
  serviceName: mysql      # 對應 Headless Service
  replicas: 1              # 多開 ≠ 資料庫叢集
  template:
    spec:
      containers:
        - name: mysql
          image: mysql:8.0.40
          envFrom:           # 整份 Secret 變成環境變數
            - secretRef:
                name: mysql-secret
          readinessProbe:    # 真的登入 ping 一次
            exec:
              command:
                - sh
                - -c
                - mysqladmin ping -h 127.0.0.1 -uroot -p"$MYSQL_ROOT_PASSWORD"
          livenessProbe:
            tcpSocket: { port: mysql }
            initialDelaySeconds: 60 # initdb 實測要 90 秒
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
  volumeClaimTemplates:    # 每個 Pod 一顆專屬 PVC
    - metadata:
        name: data          # → 產生 data-mysql-0
      spec:
        accessModes: [ReadWriteOnce]
        storageClassName: rook-ceph-block
        resources: { requests: { storage: 5Gi } }
```

StatefulSet vs Deployment

固定的 Pod 名稱 mysql-0、穩定的 DNS，Pod 重建後會綁回同一顆 PVC。

volumeClaimTemplates

PVC 的範本：每個 Pod 自動得到 data-mysql-0，不用另外寫 PVC 檔。

liveness 要給夠時間

太早探測會在 initdb 中途砍掉容器，帳號只建一半，只能刪 PVC 重來。

這裡不能 drop ALL

entrypoint 以 root 啟動再 setuid 降權，收掉 capabilities 會出現 setgid: Operation not permitted。

initContainer：先把空 PVC 準備好

07-flarum-deployment.yaml (節錄)

■ 黃字 = 本站新東西

```
spec:
  replicas: 1
  strategy:
    type: Recreate          # flarum-storage 是 RWO
  template:
    spec:
      enableServiceLinks: false # 避免 FLARUM_PORT 撞名
      initContainers:
        - name: init-storage-dirs
          image: busybox:1.36
          command:
            - sh
            - -c
            - |
              mkdir -p /storage/logs /storage/cache # 節錄
              mkdir -p /assets/avatars
              chown -R 991:991 /storage /assets
          volumeMounts:
            - name: storage
              mountPath: /storage
            - name: assets
              mountPath: /assets
      containers:
        - name: flarum
          image: mondedie/flarum:latest
          volumeMounts: # 只掛資料子目錄
            - name: storage
              mountPath: /flarum/app/storage
            - name: assets
              mountPath: /flarum/app/public/assets
      securityContext:
        fsGroup: 991
```

initContainer：先跑完才啟動主容器

用 busybox (root) 把空 PVC 的目錄建好，並 chown 給 991，主容器降權後才寫得進去。

不要整包掛 /flarum/app

Docker 會把映像內容複製進新的 volume，K8s 不會：PVC 掛上去就蓋掉程式碼，所以只掛 storage 與 assets。

enableServiceLinks: false

K8s 預設會把 Service 注入成 FLARUM_PORT=tcp://... 環境變數，跟映像自己的 FLARUM_PORT 撞名，導致 nginx 起不來。

NetworkPolicy：namespace 內的服務隔離

```
# 10-networkpolicy.yaml: mysql 的 ingress
```

■ 黃字 = 本站新東西

```
metadata:
  name: allow-ingress-to-mysql-from-flarum
spec:
  podSelector:           # 規則套在 mysql 身上
    matchLabels:
      app: mysql
  policyTypes:
    - Ingress
  ingress:
    - from:
      - podSelector: # 只收 flarum
        matchLabels:
          app: flarum
    ports:
      - protocol: TCP
        port: 3306
```

```
# 10-networkpolicy.yaml: flarum 的 egress
```

```
metadata:
  name: allow-egress-flarum-to-mysql
spec:
  podSelector:           # 規則套在 flarum 身上
    matchLabels:
      app: flarum
  policyTypes:
    - Egress
  egress:
    - to:
      - podSelector: # 只能連 mysql
        matchLabels:
          app: mysql
    ports:
      - protocol: TCP
        port: 3306
```

兩邊都要開

ingress 跟 egress 分開管制：mysql 允許進站，flarum 也要允許出站，少一條都連不上。

podSelector：namespace 內的隔離

同一個 namespace 裡的其他 Pod 一樣連不到 3306，只有 app=flarum 可以。

還有一條給 cloudbeaver

allow-ingress-to-mysql-from-cloudbeaver 用 namespaceSelector 跨 namespace 放行，第 7 站說明。

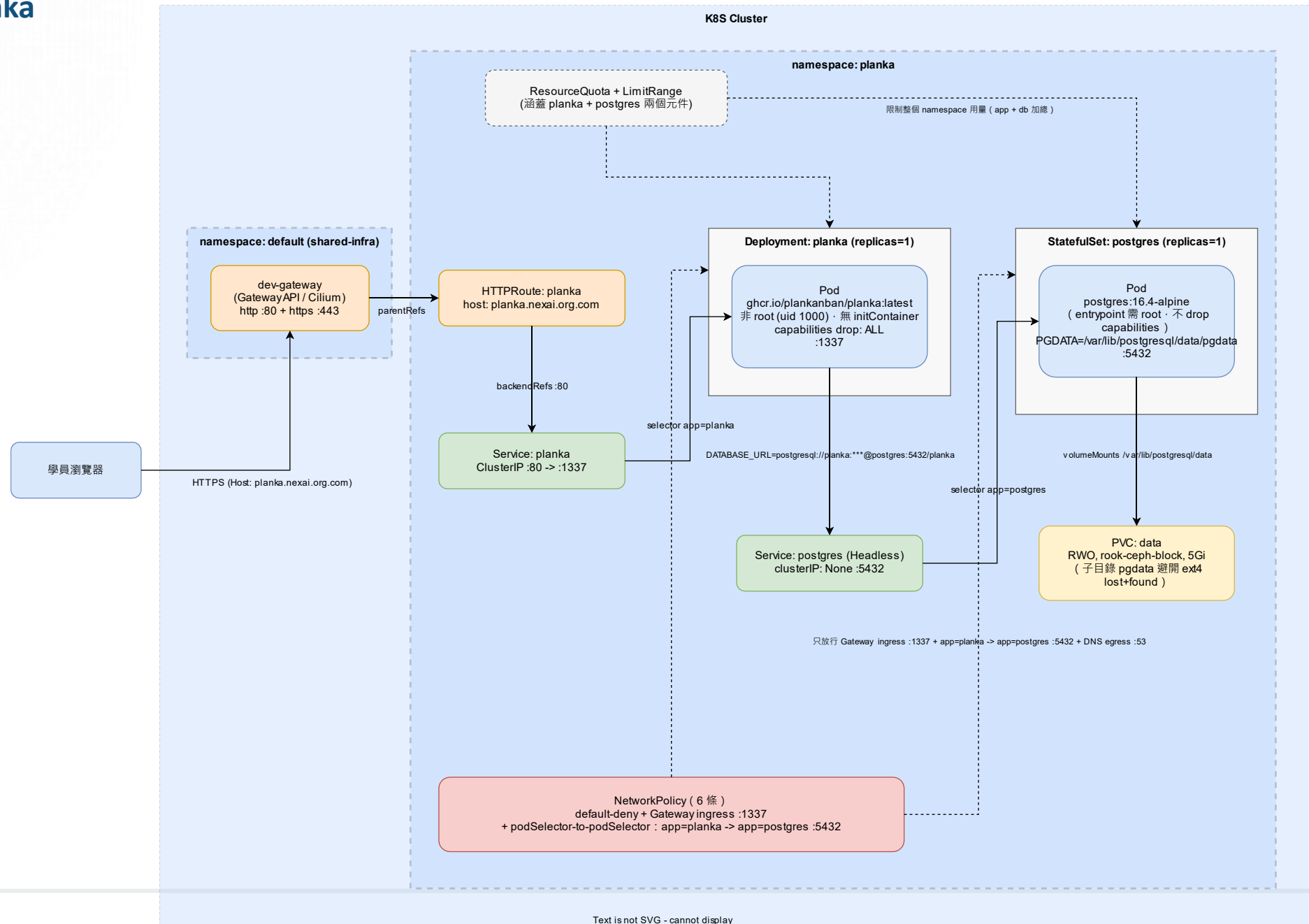
第 4 站 · plank

單元 2-4 同樣架構，換成 PostgreSQL

是開源、輕量的看板式專案管理工具。（ ghcr.io/plankanban/planka 映像 ）



第 4 站 · planka 架構圖



跟 flarum 不一樣的地方

05-planka-secret.yaml (節錄)

■ 黃字 = 本站新東西

```
stringData:
  DATABASE_URL: "postgresql://planka:TrainingPlanka123@postgres:5432/planka"
  SECRET_KEY: "TrainingOnlySecretKey_ChangeMeInProduction"
```

07-planka-deployment.yaml (節錄)

```
spec:
  replicas: 1          # 沒寫 strategy → RollingUpdate
  template:
    spec:              # 沒有 initContainers
      containers:
        - name: planka
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef: { name: planka-secret, key: DATABASE_URL }
          securityContext:
            capabilities:
              drop: [ALL]  # 非 root 映像，可以全收
```

04-postgres-statefulset.yaml (節錄)

```
env:
  - name: PGDATA      # 指到子目錄，避開 lost+found
    value: /var/lib/postgresql/data/pgdata
volumeMounts:
  - name: data
    mountPath: /var/lib/postgresql/data
```

DATABASE_URL：一個字串

帳號、密碼、主機、port、DB 名稱全塞進一個連線字串放在 Secret；flarum 則是拆成 DB_HOST / DB_USER / DB_PASS。

非 root 映像：可以 drop ALL

planka 本身以 uid 1000 執行，不需要 chown 或降權，所以也不需要 initContainer。

PGDATA 指到子目錄

Ceph RBD 格式化後，根目錄會有 lost+found；initdb 看到目錄不是空的就拒絕啟動。

RollingUpdate 回來了

planka-data 是 RWX (rook-cephfs)，新舊 Pod 可以短暫同時掛載。

第 5 站 · onlyoffice

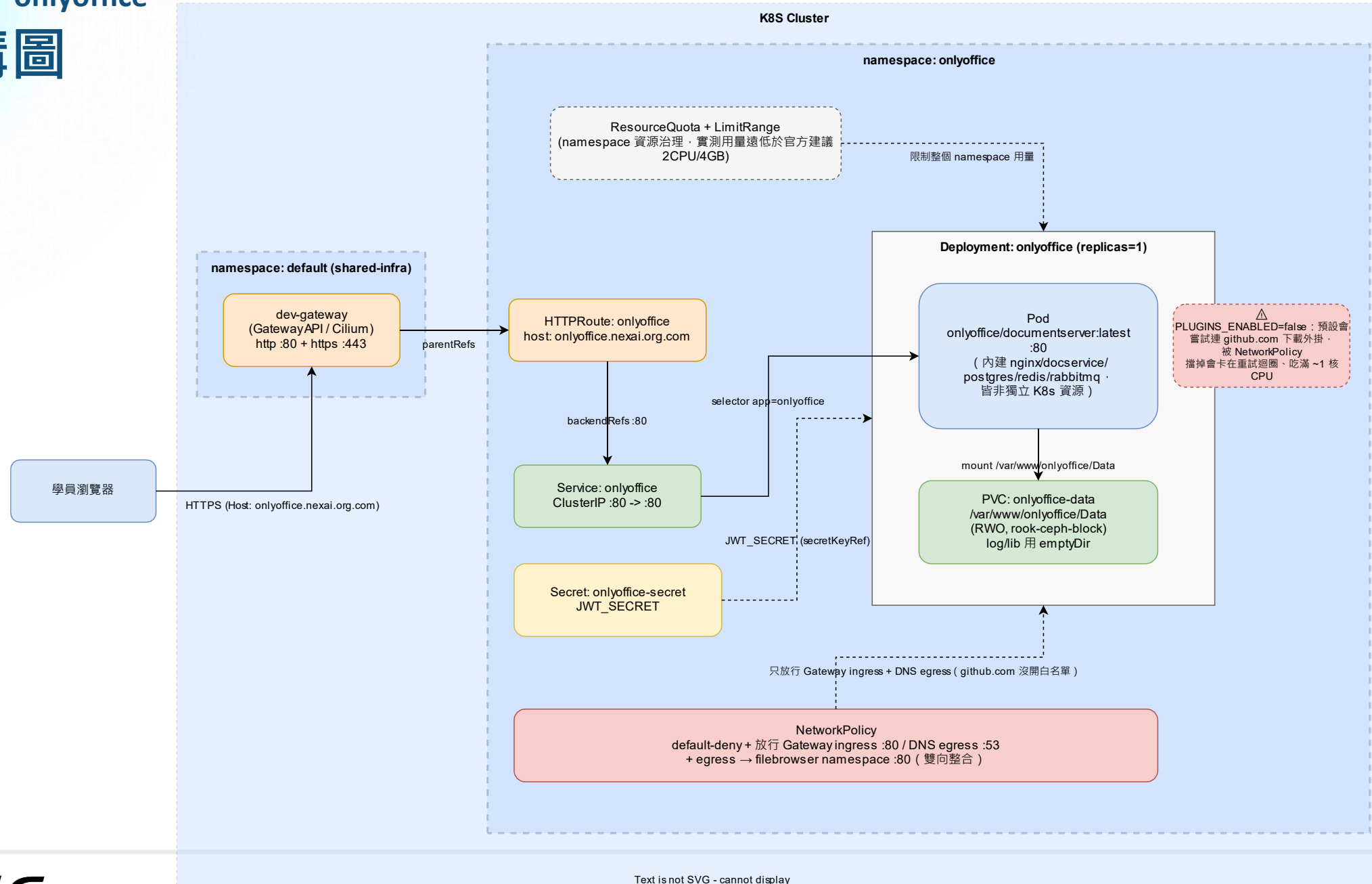
單元 2-5 重量級工作負載

是強大的線上 Office 辦公套件，支援 Word、Excel、PPT 的線上即時共同編輯，可與 Nextcloud、Filebrowser 等系統整合，官方提供「單容器全包」映像

（onlyoffice/documentserver），內建自己的 PostgreSQL、Redis、RabbitMQ，全部包在同一個 container 裡執行，不需要外接資料庫。



第5站 · onlyoffice 架構圖



資源規劃：toleration 與 nodeSelector

04-deployment.yaml (節錄)

■ 黃字 = 本站新東西

```
spec:
  replicas: 1
  strategy:
    type: Recreate
  template:
    spec:
      # nodeSelector:           # 要「釘」到 gpu01 才打開
      # kubernetes.io/hostname: gpu01
      tolerations:             # 「允許」排到 gpu01
      - key: nvidia.com/gpu
        operator: Equal
        value: "true"
        effect: NoSchedule
    containers:
      - name: onlyoffice
        image: onlyoffice/documentserver:latest
        resources:
          requests:
            cpu: "1"           # 官方建議值的一半
            memory: 2Gi
          limits:
            cpu: "2"           # 官方最低需求
            memory: 4Gi
        readinessProbe:
          httpGet: { path: /healthcheck, port: http }
          initialDelaySeconds: 60 # 內建多個服務要陸續啟動
          failureThreshold: 8
        livenessProbe:
          httpGet: { path: /healthcheck, port: http }
          initialDelaySeconds: 120
```

taint 與 toleration

gpu01 有 nvidia.com/gpu=true:NoSchedule；沒有對應 toleration 的 Pod 永遠排不上去。key / value / effect 要完全一致。

toleration ≠ 指定節點

toleration 只代表「可以」去，不會把 Pod 吸過去；要釘節點還得加 nodeSelector (或 nodeAffinity)。

Pending : Insufficient memory

k8s01 ~ 03 記憶體常在八成以上。卡住時把 nodeSelector 打開，釘到較空的 gpu01。

Quota 跟著放大

單一容器就要 4Gi limit，LimitRange.max 與 Quota limits.memory 都調到 6Gi 才放得下。

JWT、外掛開關與 emptyDir

```
# 02-secret.yaml
```

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: Secret
metadata:
  name: onlyoffice-secret
  namespace: onlyoffice
stringData:
  JWT_SECRET: "TrainingOnlyofficeJwtSecret_ChangeMe"
```

```
# 04-deployment.yaml ( env 節錄 )
```

```
env:
- name: JWT_ENABLED
  value: "true"
- name: JWT_SECRET
  valueFrom:
    secretKeyRef:
      name: onlyoffice-secret
      key: JWT_SECRET
- name: PLUGINS_ENABLED
  value: "false" # 不在背景連 GitHub
```

JWT : 兩邊金鑰逐字一致

要等於 filebrowser config.yaml 的 office.secret，否則編輯器打不開。

```
# 04-deployment.yaml ( volumes 節錄 )
```

```
volumeMounts:
- name: data
  mountPath: /var/www/onlyoffice/Data
- name: log
  mountPath: /var/log/onlyoffice
- name: lib
  mountPath: /var/lib/onlyoffice
securityContext:
  fsGroup: 102
volumes:
- name: data # 使用者文件
  persistentVolumeClaim:
    claimName: onlyoffice-data
- name: log
  emptyDir: {} # 隨 Pod 消失
- name: lib # 內建 DB 狀態
  emptyDir: {}
```

PVC vs emptyDir

emptyDir 跟著 Pod 生命週期，Pod 刪掉就清空。log 和內建 DB 狀態可以丟，只有使用者文件放 PVC。

關掉打外網的背景工作

外掛管理員在沒有對外 egress 時會不斷重試，吃掉將近 1 核 CPU。

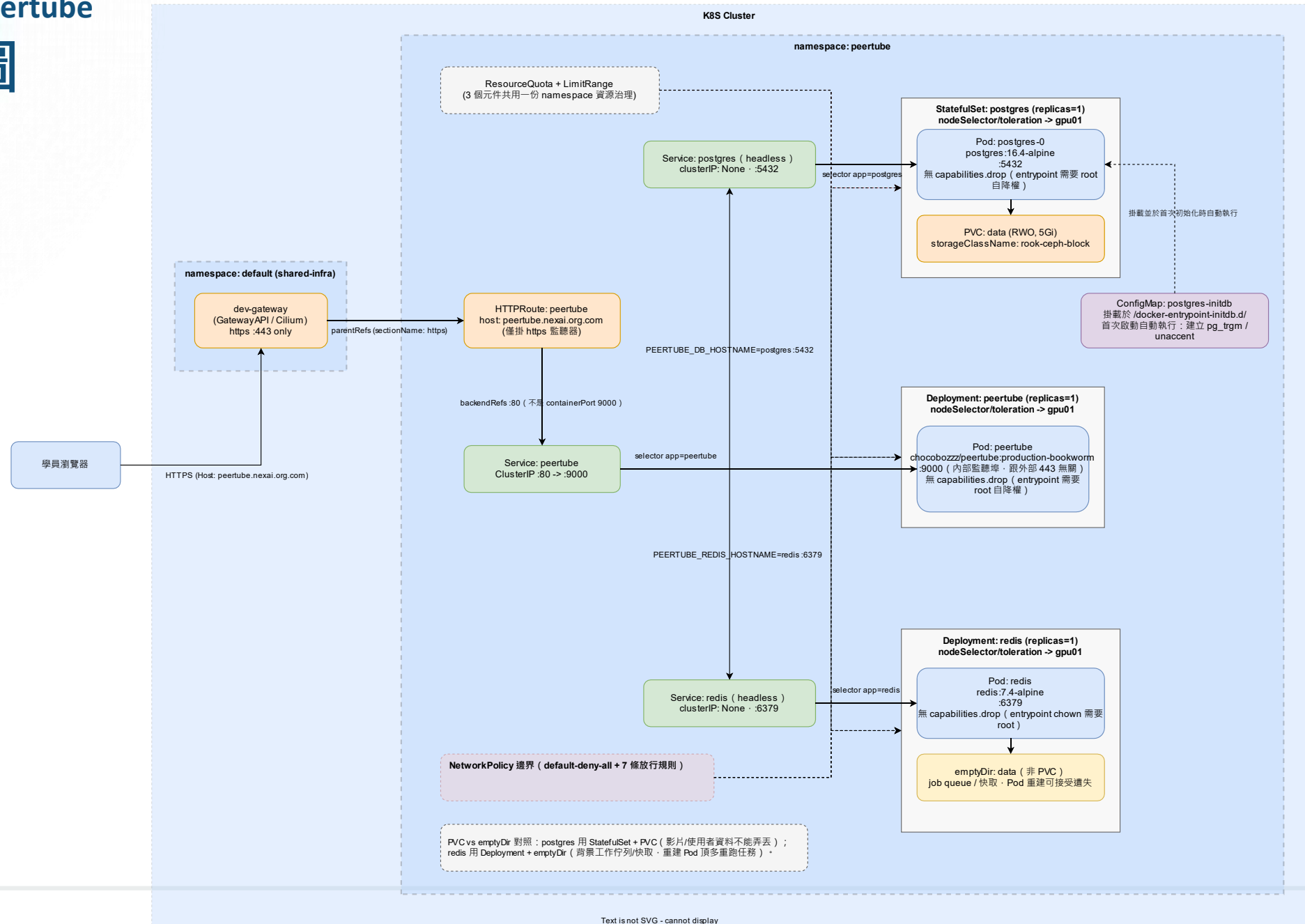
第 6 站 · peertube

單元 2-6 三個元件一起協作 (PostgreSQL, Redis, PeerTube)

是基於 ActivityPub 協定與 P2P 技術的 去中心化影音分享平台 (開源版 YouTube) 。
([chocobozzz/peertube](https://chocobozzz.github.io/peertube) 映像)



第 6 站 · peertube 架構圖



初始化腳本與 Redis 的選型

03-postgres-initdb-configmap.yaml

■ 黃字 = 本站新東西

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: postgres-initdb
  namespace: peertube
data:
  init-extensions.sql: | # key 變檔名 · 副檔名要 .sql / .sh
    CREATE EXTENSION IF NOT EXISTS pg_trgm;
    CREATE EXTENSION IF NOT EXISTS unaccent;
```

05-postgres-statefulset.yaml (節錄)

```
volumeMounts:
  - name: data
    mountPath: /var/lib/postgresql/data
  - name: initdb
    mountPath: /docker-entrypoint-initdb.d
volumes:
  - name: initdb
    configMap:
      name: postgres-initdb
```

ConfigMap 也能放初始化腳本

官方 postgres 映像第一次初始化時，會自動執行 /docker-entrypoint-initdb.d/ 底下的 .sql，不用進容器手動下指令。

06-redis.yaml (Deployment 節錄)

```
apiVersion: apps/v1
kind: Deployment # 不是 StatefulSet
metadata:
  name: redis
  namespace: peertube
spec:
  replicas: 1
  template:
    spec:
      containers:
        - name: redis
          image: redis:7.4-alpine
          readinessProbe:
            exec:
              command: ["redis-cli", "ping"]
          volumeMounts:
            - name: data
              mountPath: /data
      volumes:
        - name: data
          emptyDir: {} # 佇列 / 快取 · 可重建
```

同樣是資料庫，選型不同

postgres 存的是不能丟的資料，所以用 StatefulSet + PVC；redis 只是佇列和快取，用 Deployment + emptyDir 就好。

躲在反向代理後面的應用

09-peertube-deployment.yaml (env 節錄)

■ 黃字 = 本站新東西

```
env:
- name: PEERTUBE_WEBSERVER_HOSTNAME
  value: "peertube.nexai.org.com"
- name: PEERTUBE_WEBSERVER_PORT
  value: "443"      # 使用者看到的 port
- name: PEERTUBE_WEBSERVER_HTTPS
  value: "true"     # TLS 在 Gateway 終止
- name: PEERTUBE_TRUST_PROXY
  value: '["loopback","linklocal","uniquelocal"]'
- name: PEERTUBE_DB_HOSTNAME
  value: "postgres" # 同 namespace 短名
- name: PEERTUBE_REDIS_HOSTNAME
  value: "redis"
- name: PEERTUBE_DB_PASSWORD
  valueFrom:
    secretKeyRef:
      name: peertube-secret
      key: PEERTUBE_DB_PASSWORD
ports:
- name: http
  containerPort: 9000 # Pod 實際監聽
```

外部網址 vs 內部 port

使用者連 `https://peertube.nexai.org.com:443`，Pod 收到的卻是 9000 port 的明文 HTTP。影片網址與 ActivityPub 聯邦資訊都要用外部網址。

TRUST_PROXY

預設只信任 loopback。流量從 Gateway 的 Pod 網路（私有位址）進來，要加上 uniquelocal，才拿得到正確的用戶端 IP 與 https 判斷。

三個元件用短名互連

postgres、redis 都是同 namespace 的 Headless Service，env 直接寫 Service 名稱。

7 條 NetworkPolicy

- 1 default-deny-all**
全部 Pod：先全擋
- 2 allow-ingress-to-peertube-from-gateway**
peertube ← 任何來源 :9000
- 3 allow-ingress-to-postgres-from-peertube**
postgres ← peertube :5432
- 4 allow-ingress-to-postgres-from-cloudbeaver**
postgres ← cloudbeaver namespace :5432
- 5 allow-ingress-to-redis-from-peertube**
redis ← peertube :6379
- 6 allow-egress-dns**
全部 Pod → DNS :53
- 7 allow-egress-peertube-to-backends**
peertube → postgres :5432、redis :6379

12-networkpolicy.yaml ⑦

■ 黃字 = 本站新東西

```
metadata:
  name: allow-egress-peertube-to-backends
spec:
  podSelector:
    matchLabels:
      app: peertube
  policyTypes:
    - Egress
  egress:
    - to:          # 一條 policy 列兩個目的地
      - podSelector:
          matchLabels:
            app: postgres
      ports:
        - protocol: TCP
          port: 5432
    - to:
      - podSelector:
          matchLabels:
            app: redis
      ports:
        - protocol: TCP
          port: 6379
```

一個元件一組白名單

三個元件，每條連線都要 ingress 跟 egress 各開一次，所以有 7 條。

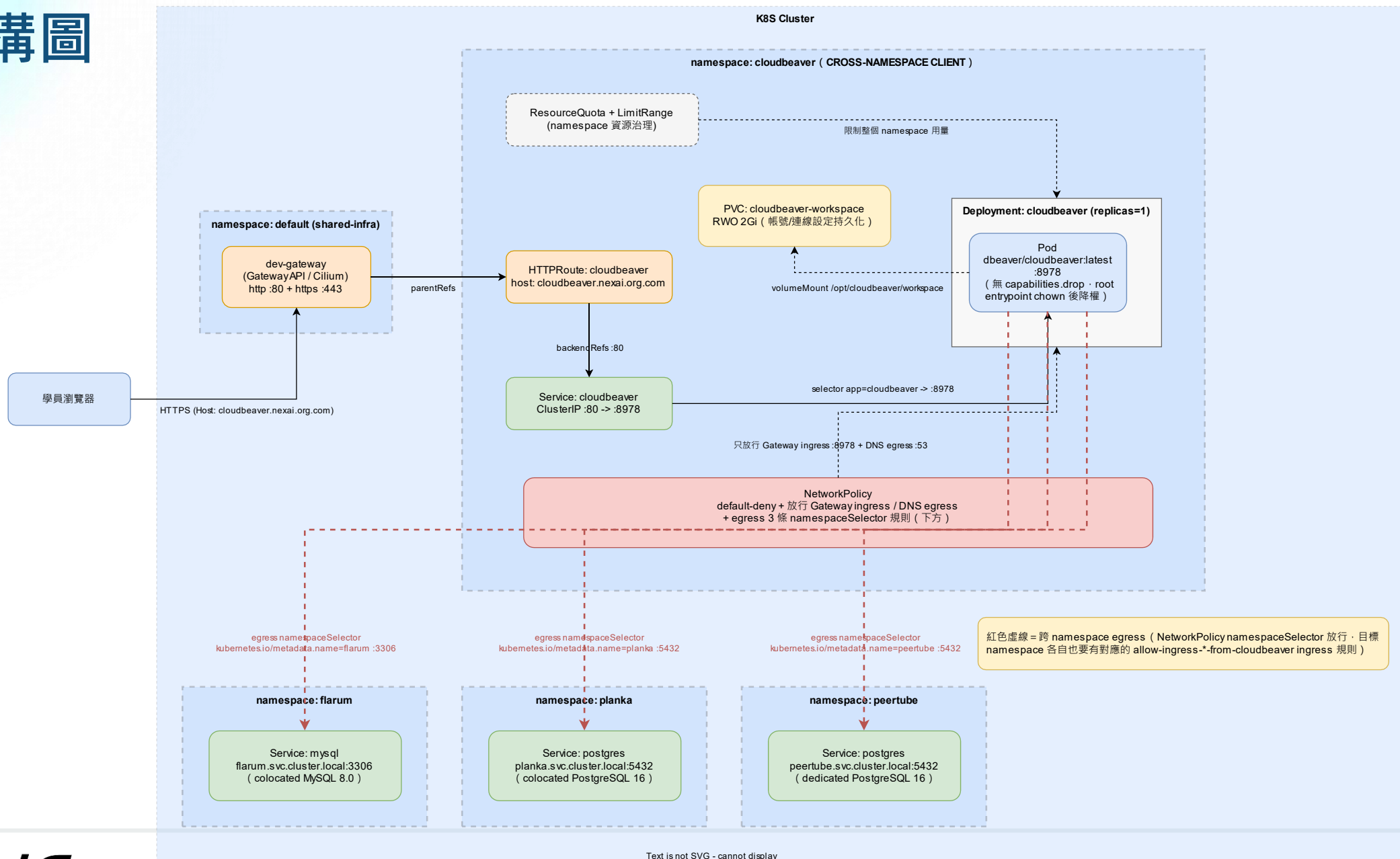
第 7 站 · cloudbeaver

單元 2-7 跨 namespace 存取

是 DBeaver 官方推出的 Web 版本，一個通用的視覺化資料庫管理工具（DBeaver-like），同一個瀏覽器介面可以建立多組資料庫連線、支援 MySQL/PostgreSQL 等多種資料庫。
(dbeaver/cloudbeaver 映像)



架構圖



跨 namespace : namespaceSelector

06-networkpolicy.yaml (cloudbeaver 端)

■ 黃字 = 本站新東西

```

metadata:
  name: allow-egress-cloudbeaver-to-databases
  namespace: cloudbeaver
spec:
  podSelector:
    matchLabels:
      app: cloudbeaver
  policyTypes:
    - Egress
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: flarum
      ports:
        - { protocol: TCP, port: 3306 }
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: plank
      ports:
        - { protocol: TCP, port: 5432 }
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: peertube
      ports:
        - { protocol: TCP, port: 5432 }
```

03-flarum/manifest/10-networkpolicy.yaml (資料庫端)

```

metadata:
  name: allow-ingress-to-mysql-from-cloudbeaver
  namespace: flarum
spec:
  podSelector:
    matchLabels:
      app: mysql
  policyTypes:
    - Ingress
  ingress:
    - from:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: cloudbeaver
      ports:
        - { protocol: TCP, port: 3306 }
```

kubernetes.io/metadata.name

K8s 自動幫每個 namespace 加上的 label，不用自己手動貼。

兩邊都要開門

cloudbeaver 開 egress；flarum、plank、peertube 各加一條 ingress。部署前要重新 apply 那三個產品的 NetworkPolicy。

03

練習與驗收

單元 3 了解各站功能與架構後，相同 7 站產品根據題目進行部署練習



兩種題型：填空與除錯

題目一 · `manifest-incomplete/`

半成品 YAML 填空

正解被挖掉的欄位標成 `__FILL_ME_n__`，照編號填完再依序 apply。

寫完用 diff 跟 `../manifest/` 對照。先自己推理一輪，再對答案。

題目二 · `manifest-buggy/`

看起來完整，其實藏了錯

整套 apply 下去，用 describe、get -o yaml、logs 找出問題。

題目說明：<https://github.com/AnnLeemocha/k8s-training/tree/main/實作練習題目.pdf>

除錯工具箱：看到症狀先查哪裡

症狀	先看哪裡	這門課常見的原因
Pod 卡在 Pending	<code>kubectl describe pod → Events</code>	超過配額、缺 toleration、PVC 未 Bound
Running 但 0/1 Ready	<code>kubectl describe pod → probe</code>	probe 打錯 port、初始延遲太短
Endpoints 是空的	<code>kubectl get endpoints</code>	selector 跟 Pod label 對不上
Gateway 回 503	<code>kubectl describe httproute</code>	backendRefs 不是 Service 的 port
CrashLoopBackOff	<code>kubectl logs --previous</code>	drop ALL 讓降權失敗、設定路徑錯
連線逾時	<code>kubectl exec 測 /dev/tcp</code>	NetworkPolicy 少放行一個方向

驗收標準：四個綠燈才算部署完成

✓ 1

Pod Running 且 Ready

```
kubectl get pod -n <ns>
```

✓ 2

Service 有 endpoints

```
kubectl get endpoints -n  
<ns>
```

✓ 3

**HTTPRoute Accepted
+ ResolvedRefs**

```
kubectl describe  
httproute -n <ns>
```

✓ 4

curl 拿到應用的真實內容

```
curl -k  
https://<product>.nexai.  
org.com
```

單一產品收尾

```
kubectl delete namespace <product>
```

整門課結束：刪掉教學 Gateway 一次收尾

```
kubectl delete -f dev-gateway.yaml
```

The background of the slide is a photograph of a city street, possibly in New York City, with a blue overlay. The street is lined with buildings and has a yellow taxi visible. The text "Thank You" is written in a large, white, sans-serif font on the left side of the image.

Thank You

NEXT：備份、復原與災難復原演練